HTML Quality Detection for Muck Rack Article Ingestion Pipeline

Rishabh Patil
rbp5812@nyu.edu

Deepali Balakrishna Ksheersagar
db5144@nyu.edu

Kund Meghani
km6579@nyu.edu

Abstract

We present an HTML quality detection pipeline developed for Muck Rack's article ingestion system. The approach combines rule-based heuristics with an XGBoost classifier to distinguish normal from broken HTML using structural and content-derived features. The resulting system is designed for real-world deployment, enabling reliable extraction of high-quality information from web articles.

1 Introduction

Muck Rack is a platform used by PR professionals and journalists for media monitoring, contact management, pitching, and analytics. A critical component of the product is the article ingestion pipeline, which downloads webpages at scale to power search and downstream workflows.

2 Problem statement

At Muck Rack's ingestion scale, a successful HTTP response (e.g., status code 200) does not guarantee that the returned HTML is usable. In many cases, the downloaded page cannot be parsed into meaningful article content, creating *silent failures* that are difficult to detect automatically. Today, some issues are surfaced through manual editorial review, but the volume of daily ingested pages makes comprehensive manual inspection infeasible.

We aim to build an automated **HTML Quality Detection** system that classifies each downloaded page as: **Normal HTML** or **Broken HTML**.

Definition (Broken / Unqualified HTML). A page is considered broken if we cannot parse any useful information from it (e.g., empty HTML, cookie/privacy-only content, incorrect decoding, or pages that contain primarily JavaScript that dynamically loads content, including Shadow DOM patterns). Pages with *partial but meaningful* content (e.g., paywalled pages with a readable teaser or summary) are considered **Normal**.

Requirements: The solution should achieve strong precision and recall (both are important for our use cases) while maintaining an inference time suitable for processing millions of pages per day [6,7].

3 Dataset challenge: extreme class imbalance

A major early bottleneck was the labeled dataset itself. The initial labeled pool was highly imbalanced, with only ≈ 38 broken HTML pages versus ≈ 2500 normal pages. This imbalance in the dataset limited our ability to train and evaluate reliable classifiers, especially for the rare **"broken"** failure HTMLs that matter most in production.

4 GPT-assisted labeling pipeline (offline dataset expansion)

To expand labels beyond the small, highly imbalanced seed set, we implemented a GPT-based *offline labeling* pipeline. This pipeline was used only to create additional labeled examples from an additional $\sim 15{,}000$ unlabeled HTML pages provided by Muck Rack, and was not used for production inference at Muck Rack scale.

4.1 Pipeline overview

Given a folder of unlabeled HTML files, the notebook executes the following steps:

1. **GPT labeling on remaining HTMLs.**
   For each remaining HTML file, the pipeline:
   - extracts visible text using BeautifulSoup (`soup.get_text(...)`)[10],
   - computes lightweight keyword/regex features and small raw-HTML evidence snippets
   - sends a structured prompt to GPT (model set to `gpt-4.1-mini`, `temperature=0`) requiring **strict JSON** output.
2. **Outputs + file routing.**
   The pipeline writes a CSV (`GPT_label.csv`) containing `link_id`, predicted label, confidence, matched cues, and a short rationale. It also *moves* each processed HTML into folders based on the result:
   - `Qualified_htmls`: label `qualified_content`
   - `Error_htmls`: labels that include (e.g., cookie-only, 404, JS error, dynamic shell, undecoded text, etc.)

4.2 Prompt template (system rules + few-shot examples)

The GPT call uses a **system instruction** that forces the model to choose *exactly one* label from: {`cookie_banner`, `javascript_error`, `404_page`, `dynamic_app_shell`, `undecoded_text`, `qualified_content`, `empty`}.

It also enforces a **priority order** (e.g., 404 cues override cookie cues; cookie-only overrides JS-only; JS-only overrides dynamic shell; etc.) and explicitly states that **paywalled pages with meaningful teaser/summary text are still** `qualified_content`.

To stabilize outputs, the notebook includes **few-shot examples** for canonical cases (cookie banner, JS-required, 404, dynamic app shell, undecoded text, qualified content). The user message payload is a JSON object containing:

- `merged_text`: extracted visible text (with line breaks preserved),
- `features`: hit counts for cookie/JS/404/dynamic/bot patterns, an "undecoded" ratio, and a few short raw HTML snippets around matched patterns (to give GPT concrete evidence).

The assistant must return strict JSON:

$$\{\texttt{label},\ \texttt{confidence} \in [0,1],\ \texttt{matched_cues},\ \texttt{rationale}\}.$$

If parsing fails or the label is invalid, the code defaults to `unknown`.

Using this GPT-assisted labeling pipeline for dataset expansion, we increased our labeled dataset from the initial seed set to roughly $\sim 17{,}500$ samples, including approximately $\sim 1{,}500$ broken HTML pages.

4.3 Why GPT was not used for production inference

Although effective for quickly enriching labels, running every ingested HTML through GPT would not be economically feasible at Muck Rack's scale (millions of pages per day), and would introduce avoidable latency and external service dependencies. Therefore, GPT was used only to expand the labeled dataset, after which we focused on building a lightweight, scalable model for inference.

5 Approach to Classification

5.1 Flowchart

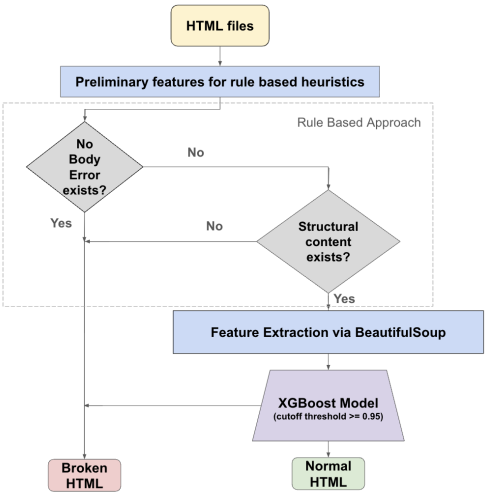

Figure 1: Flowchart

5.2 Rule Based Heuristics

Before training a statistical model, we implemented a fast rule-based screening layer to detect the most common *obviously unqualified* HTML failures and to reduce noise in the downstream modeling pipeline. These heuristics are designed to be computationally cheap (single-pass parsing + simple regex/length checks), making them suitable for high-throughput processing while also providing interpretable signals about why a page is considered broken[8].

5.2.1 Preprocessing: fast visible-text extraction

We first extract a proxy for the page's visible text by removing `<script>` and `<style>` elements and then joining the remaining text nodes. This prevents JavaScript-heavy pages and embedded styling from inflating word counts and improves the reliability of text-based heuristics. Word counts are computed using a lightweight token regex (`\w+`) for speed and robustness. HTML parsing and feature extraction were implemented using `lxml` for efficiency [9].

5.2.2 Structural content detection

A central heuristic checks whether the HTML contains evidence of "article-like" structure. Concretely, we mark a page as structurally qualified if *any* of the following signals are present:

- **Informative metadata:** a sufficiently long `meta` description (e.g., `name=description` or `property=og:description`) indicating meaningful content.
- **Semantic containers:** non-trivial text under `<article>` or `<main>` tags.
- **Content-like identifiers:** `div/section/p` elements whose `class` or `id` matches content-oriented patterns (e.g., `content`, `article`, `story`, `news`) and also contain substantial text.
- **Paragraph density:** a sufficient number of paragraphs and total paragraph length, capturing cases where the page is text-heavy even without clean semantic tags.
- **Table-based fallbacks:** large `<td>` text blocks to handle legacy layouts where article content is embedded in tables.

To avoid false positives from navigation-only pages, we additionally require a minimum amount of visible text. Pages with a very low visible word count are treated as structurally unqualified. If none of these signals are present in the HTML page, then the HTML is classified as **"Broken."**

5.2.3 No-body / empty-page detection

We also include a targeted heuristic to detect "no-body" failures, where the downloaded HTML contains little to no extractable body content. Using the visible text proxy, we compute:

- total visible words in the document,
- words inside `<body>`, and
- words inside `<head>`.

A page is flagged as **"Broken"** when the body text is below a certain threshold, while overall visible text remains small, indicating that the download likely returned an empty shell, or a non-article page.

Overall, this rule-based layer catches clear-cut broken cases early with minimal compute.

5.3 Modeling

While the rule-based layer is effective for catching obvious failures, many real-world ingestion issues are more subtle and cannot be reliably identified using rule-based alone. To address these borderline cases, we introduced a supervised modeling stage that learns a decision boundary from labeled examples, combining the heuristic signals (e.g., word-count and structural indicators) with additional content- and markup-derived features. This model produces a probability of an HTML being `broken` versus `normal`.

5.3.1 EDA

The dataset provided by Muck Rack consisted of a `link_id`, URL, page title, textual content, ground-truth label, and the complete corresponding HTML document. Feature extraction was performed across three primary sources: the URL (For URL-based retrieval and testing, pages were fetched using the `requests` library [14]), the HTML structural properties, and the visible page content.

A diverse set of features was initially extracted, including structural indicators such as tag counts and DOM characteristics, as well as content-based statistics:

```
[num_tags_total, num_script_tags, num_link_tags, num_img_tags,
num_a_tags, num_meta_tags, max_dom_depth, doctype_present_flag,
missing_body_flag, missing_head_flag, has_404_text_flag,
tag_to_text_ratio, has_script_overload, text_length, word_count,
unique_word_ratio, avg_word_length, num_digits, num_special_chars,
title_length, empty_text_flag, empty_title_flag]
```

Correlation-based feature analysis revealed that only a subset of these features contributed meaningfully to model performance. Empirically, the classifier achieved better results when relying primarily on numerical signals derived from HTML structure and lightweight content statistics, rather than raw textual features. This approach also significantly reduced computational overhead.

5.3.2 Feature Selection

Further, The Feature Selection was done using *Leave-One-Feature-Out* (LOFO) analysis, resulting in the removal of multiple low-impact features and a more compact feature set[12].
Features Used: The extracted features are grouped into *structural* and *content-based* categories to capture both HTML layout integrity and textual quality.

Structural Features

- **num_tags_total**: Total number of HTML elements on the page.
- **missing_head_flag**: Binary indicator of whether the `<head>` tag is missing.
- **script_tag_ratio**: Fraction of `<script>` tags relative to all HTML tags.
- **empty_title_flag**: Binary indicator of whether the `<title>` tag is empty or missing.
- **text_html_ratio**: Ratio of visible text size to raw HTML size.

Content-Based Features

- **text_length**: Total number of visible text characters.
- **special_chars**: Count of punctuation and symbol characters in visible text.
- **num_digits**: Number of numeric characters in visible text.
- **digit_ratio**: Proportion of digits relative to total visible text length.
- **max_paragraph_length**: Length of the longest detected text paragraph.
- **num_sentences**: Count of sufficiently long, sentence-like text spans.
- **avg_sentence_length**: Average character length of detected sentences.

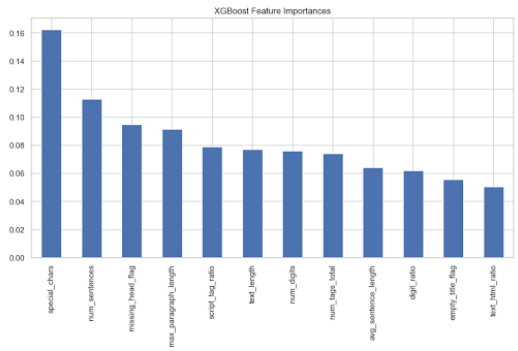

Figure 2: Feature importance

5.3.3 Model Selection

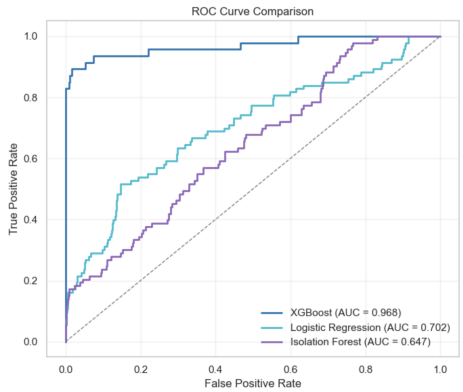

Figure 3: Flowchart

We evaluated both *XGBoost* (non-parametric) and *logistic regression* (parametric) supervised learning models. In addition, an *Isolation Forest* model [3] was explored to compute anomaly scores, both as a standalone detector and as an auxiliary feature combined with XGBoost[1]. However, experimental results indicated that XGBoost performed optimally without the inclusion of Isolation Forest-based anomaly scores, leading to their exclusion from the final pipeline. Hyperparameters were tuned using Optuna on validation performance [2].

6 Final Results For XGBoost Model

We evaluate the final XGBoost classifier [1] on a held-out test set of 2,679 unseen HTML pages from the Muck Rack corpus. The holdout split preserves the strong class imbalance observed in production, with broken HTML pages forming a small minority of the data. Inference-time profiling was performed using Python's built-in `cProfile` module [13].

Function	Mean Calls	Mean Time (s)	Cumulative Time (s)
process_one_html_full	1.0	0.000062	0.032132
extract_all_features	1.0	0.000198	0.025701
classify_n_files_for_profile	0.01	0.000316	0.032705
feed	1.0	0.002404	0.017587
handle_starttag	752.0	0.000866	0.006619
endData	2313.0	0.001269	0.004028

Table 1: Inference profile results of major functions in the HTML quality detection pipeline.

6.1 Classification Performance

At the selected operating threshold of 0.954, chosen to maximize the F1-score on validation data, the model achieves strong overall performance [6,7]. The classifier attains an overall accuracy of 0.99, with a macro-averaged F1-score of 0.93 and a weighted F1-score of 0.99.

Class-wise performance is summarized as follows:

- **Normal HTML (Class 0):** Precision = 0.99, Recall = 1.00, F1-score = 1.00 (support = 2,586)
- **Broken HTML (Class 1):** Precision = 0.91, Recall = 0.83, F1-score = 0.87 (support = 93)

6.2 Operating Point and Error Analysis

At the chosen threshold, the resulting confusion matrix is:

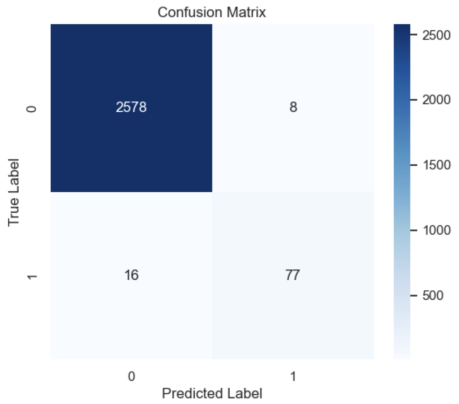

Figure 4: Confusion Matrix

This operating point corresponds to a precision of 0.906 and a recall of 0.828 for the broken HTML class, yielding a best F1-score of 0.901. Only eight false positives are observed among more than 2,500 normal pages, indicating a very low false-alarm rate. While 16 broken pages are missed, this trade-off was intentionally selected to prioritize precision.

7 Conclusion

This paper presents a practical, scalable approach for detecting broken or unqualified HTML in large-scale web ingestion, where a successful HTTP response does not necessarily imply extractable content. To overcome severe class imbalance and limited labels, we expand the dataset using a structured GPT-assisted offline labeling pipeline and deploy a two-stage system: a fast rule-based screening layer for obvious failures, followed by an XGBoost classifier trained on lightweight structural and text-statistic features. On an unseen holdout set of 2,679 pages, the model achieves 0.91 precision and 0.83 recall for the broken class (F1 = 0.87) at the chosen threshold with very few false positives. Overall, the approach is accurate and deployment-friendly, improving downstream

extraction reliability and data quality, with future work focused on monitoring new failure modes, periodic retraining, and cost-aware threshold tuning.

Acknowledgements

We sincerely thank our industry mentors at Muck Rack—Mari Pastor, Matt Dzungan, Ruochen Zhong, and Franklin Liu—for their continued guidance, technical feedback, and support in shaping the problem definition and ensuring the practical relevance of our approach. We are also grateful to the NYU capstone mentors and course staff—Aahlad Manas Puli ,Shauli Ravfogel, Byung-Doh Oh, Hongjun Liu, Yihuai Hong, Yuxuan Wang, and Dennis Hu—for their mentorship, constructive suggestions, and ongoing encouragement throughout the project.

References

[1] Chen, T. & Guestrin, C. (2016) XGBoost: A scalable tree boosting system. In B. Krishnapuram, M. Shah, A. Smola, C. Aggarwal, D. Shen, & R. Rastogi (eds.), *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 785–794.

[2] Akiba, T., Sano, S., Yanase, T., Ohta, T. & Koyama, M. (2019) Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*.

[3] Liu, F.T., Ting, K.M. & Zhou, Z.-H. (2008) Isolation Forest. In *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM)*, pp. 413–422.

[4] Lundberg, S.M. & Lee, S.-I. (2017) A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[5] Ratner, A., Bach, S.H., Ehrenberg, H., Fries, J., Wu, S. & Ré, C. (2020) Snorkel: Rapid training data creation with weak supervision. *The VLDB Journal*.

[6] Davis, J. & Goadrich, M. (2006) The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*.

[7] Saito, T. & Rehmsmeier, M. (2015) The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*.

[8] Kohlschütter, C., Fankhauser, P. & Nejdl, W. (2010) Boilerplate detection using shallow text features. In *Proceedings of the Third ACM International Conference on Web Search and Data Mining (WSDM)*.

[9] lxml. (n.d.) lxml: XML and HTML processing in Python.
`https://lxml.de/` (accessed Dec. 17, 2025).

[10] Richardson, L. (n.d.) Beautiful Soup documentation.
`https://beautiful-soup-4.readthedocs.io/` (accessed Dec. 17, 2025).

[11] Reitz, K. & Contributors. (n.d.) Requests: HTTP for Humans.
`https://requests.readthedocs.io/` (accessed Dec. 17, 2025).

[12] Aerdem, "LOFO Feature Importance."
`https://github.com/aerdem4/lofo-importance` (accessed Dec. 16, 2025).

[13] Python Software Foundation. (n.d.) `cProfile` — The Python profiler.
`https://docs.python.org/3/library/profile.html` (accessed Dec. 17, 2025).